Toward World Models of Organizations: Every System Sees the Same Company

Ashwin Gopinath
Sentra, San Francisco

November 2026

Abstract

A company is one world observed by many systems. Email, chat, the CRM, support tickets, call recordings, calendars and issue trackers each record part of what is happening with every customer, deal and project, yet no system records the situation itself. People rebuild it in their heads; software stores disconnected records. We ask how a machine can learn the joined-up picture: a *world model of the organization* whose state space places every record by the situation it describes, whichever system wrote it. Our starting point is the Platonic Representation Theorem, which shows that joint-embedding predictive architectures (JEPAs) trained on *good views*, different observations of one underlying state, converge to the same representation up to a rotation. Our observation is that organizations already keep good views without anyone annotating them: records that different systems link to the same customer in the same time window. On a simulated company from Era by Eon (seven systems, 21,972 customer-linked records, exact ground truth), we train small text JEPAs from scratch on such links and compare them with corruption-based views and next-token prediction on identical text. On records whose links were hidden during training, good views place 76% of records with the right customer, against 53% for TF-IDF, the strongest alternative. With customer names and identifiers masked out of every record, about half of the customer signal remains. Encoders trained separately on messages and on systems of record can be partly aligned by one rotation fitted on 64 anchor pairs, unlike the alternatives, and a rotation lets a retrained encoder search the index built by the old one (79% of nearest neighbours on the right customer, against 3% without it). We also build access control into the model. Secrets planted in records a reader may not read leak through an encoder trained on everything even when search results and states are filtered to readable records (a codename search still finds the right customer 100% of the time). With access control built in, what each reader receives depends only on what they may read, and the planted secrets change nothing. We also report what does not work yet: the state vectors do not forecast the next quarter, and they are poor at facts about which system recorded what. We argue that every company will come to run on an operational world model of this kind, which people and agents can understand the company through, learn on and improve, and we release our code and data exporter.

1 Introduction

Ask anyone in a company what is going on with its largest customer. The honest answer is spread across a sales call last Tuesday, three support tickets, a chat thread, an email chain with procurement and a CRM stage that is two weeks out of date. Each system records a piece of the situation; none records the situation itself. People rebuild it in their heads, slowly, partially and differently from one another. AI assistants rebuild it from scratch every time they are asked, by retrieving documents and reading them.

What is missing is a model of the company itself: a *world model*. In robotics and video, a world model is a learned representation of the state of the world together with a way to predict how that state

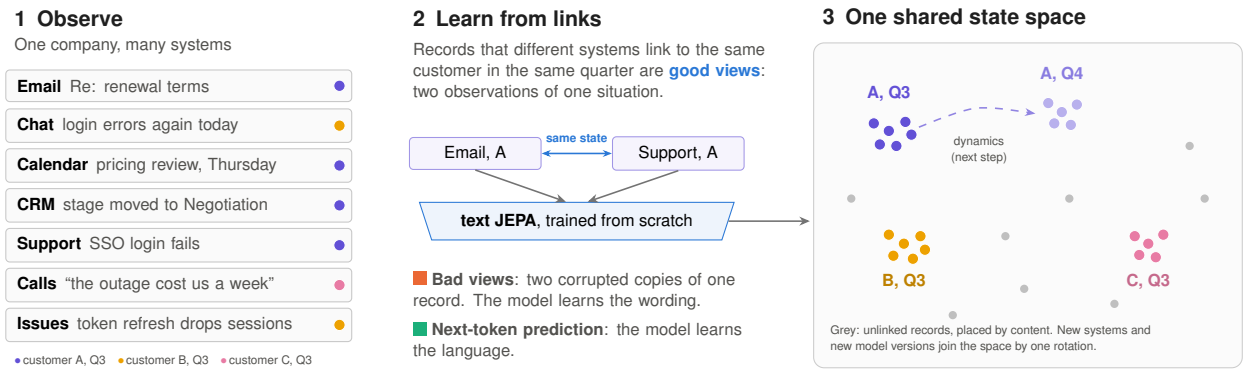


Figure 1: The idea. (1) Every system records a piece of each customer's situation. (2) The links the systems already keep tell us which records are two observations of the same situation, the *good views* that the Platonic Representation Theorem asks for. (3) A JEPA trained on them maps records from every system into one state space where records about the same customer and quarter land together and records without links can be placed by content; separately trained or retrained encoders can be partly aligned by a rotation. Predicting how states move is the next step toward a full world model.

changes [12, 13, 20]. For an organization, the world is the situation of every customer, deal, project and team at every moment, and the observations are the records the organization produces as it works. A world model of an organization would map every record, from every system, into one state space where records about the same situation land together. It would stay stable as systems, models and time change. Eventually, it would predict how states evolve and how actions change them.

We think every company will come to run on such a model: an *operational world model* that represents how the company actually functions. A capable new employee spends months learning a company from what happens around them: which customers are struggling, which deals are moving, who knows what. An operational world model lets people and AI agents learn the company the same way, from the company's own record of what happened, and lets them try a change against the model before making it in the company. Understanding the company, learning on it, and improving it all start from the same object.

The hard part is learning such a space without labels. Nobody annotates "situations". Generic text embeddings organize records by topic and template; next-token prediction learns the language of the records, not the company. The Platonic Representation Theorem [3] makes the requirement precise. JEPAs trained on **good views**, two different observations of the same underlying state, converge to the same representation up to one rotation, whatever the modality. With *bad views*, corruptions of a single observation, they do not, and next-token prediction behaves like a JEPA with bad views. In vision and language, good views are scarce: someone has to pair an image with its caption.

A model of a company has a requirement that a model of the physical world does not: not everyone may see everything. Sales notes, support escalations, leadership discussions and private messages each have their own readers, and a model that learns from all of them can repeat what it learned to anyone who asks. Access control therefore has to be part of the model, not a filter applied to its output.

Our central observation is that **organizations produce good views for free**. The same customer appears in an email, a ticket, a call and a CRM update in the same week, and the systems themselves record the links: account identifiers, deal identifiers, organization fields, channel names, calendar invitations. Records linked to the same entity in the same time window are different observations of the same situation. The links a company already keeps are, in the language of the theorem, a supply of good views.

We test this on a simulated company generated by Era by Eon [8, 11], whose system simulators produce one consistent company with exact ground truth. We train small text JEPAs from scratch on the company's

own records and compare good views, bad views and next-token prediction on identical text, architecture and budget. Our contributions are:

- A definition of an organizational world model and a recipe for its state space: states are (entity, time window) pairs, observations are records from any system, and good views are records linked to the same state (§2, §4).
- Evidence that the links make the space. On records whose links were hidden during training, good views place 76% of records with the right customer, against 53% for TF-IDF and 19% and 20% for bad views and next-token prediction (§6.2). On the training records, 95% of a record's nearest neighbours from other systems concern the same customer, against 13–14% for the alternatives (§6.1). About half of that signal survives masking every customer name and identifier (§6.3), and it degrades gracefully as links are corrupted (§6.4).
- Evidence that the space can be shared and updated. Encoders trained separately on different systems can be partly aligned by one rotation fitted on 64 anchor pairs, and encoders trained with the alternatives cannot (§6.5). A retrained encoder can search the old index through one rotation, although only the leading directions of the space survive retraining (§6.6).
- Access control built into the world model. We identify three ways a learned world model leaks across access boundaries (search results, aggregated states and the training of the encoder itself) and close all three, so that what each person receives depends only on records they may read. Planted secrets that leak through the training data even when search is filtered do not leak through the built-in design, which is non-interfering by construction (§4, §6.7). To our knowledge, this is the first world model of an organization with access control built in.
- An account of what does not work yet, forecasting and the facts that depend on which system wrote a record, and what it implies for the rest of an organizational world model (§6.8, §7).
- Code, an exporter for Era's simulators and all results (§10).

Context. At Sentra we are building operational world models of companies, on top of an organizational memory that captures how work happens across a company's systems. This paper describes building blocks of that effort, learning a shared state space and building access control into it. To keep the work reproducible and independent of any product, we study it on a public simulator.

2 What is a world model of an organization?

States, observations and dynamics. We model an organization as a set of entities e (customers, deals, projects, people) whose situation changes over time. Write $s_{e,t}$ for the latent state of entity e during time window t: everything that is true about that customer this quarter, from the open issues and the deal stage to the people involved and the mood of the last call. States are never observed directly. What the organization observes are *records* x: an email, a chat message, a ticket comment, a call-transcript turn, a CRM change, an issue, a meeting. Each record is rendered from a state by a system and carries nuisance,

$$x = r_m(s_{e,t}, n), \tag{1}$$

where m is the system (email, chat, CRM, ...) and n collects everything that is not the situation: the author, the template, the wording, the formatting. A world model of the organization has three parts:

(i) an **encoder** f that maps any record into a shared state space, $f(x) \approx g(s_{e,t})$, invariant to the system m and the nuisance n;

(ii) a **memory** that aggregates encoded records into current states and serves them to people and programs;

(iii) a **predictor** that maps a state, and possibly an action, to the next state: $s_{e,t+1} \approx h(s_{e,t}, a_t)$.

This paper is about the first part, the state space, and about keeping it stable as the organization changes. We report early and mostly negative results on the third.

What a shared state space is for. It turns everyday questions into geometry. *Where are we?* The state of a customer this week is a point, and its nearest neighbours from every system are the records that matter. The encoder can also place records that nobody linked, which in our simulated company are 72% of all records; we test this by hiding links (§6.2). *What changed?* The difference between this week's state and last week's. *What is this like?* Situations that look alike across customers, such as accounts that look the way a lost account looked before it was lost. *Is something off?* A state that moves fast, or two teams' records about the same customer drifting apart. *What about new tools and new models?* In our experiments, separately trained encoders can be partly aligned by one rotation fitted on 64 matched records, and a retrained encoder can search the old index through another (§6.5, §6.6).

For an individual the difference is concrete. A new account manager sees one customer's situation assembled from systems they have never opened. A support engineer sees that the ticket in front of them belongs to a customer whose renewal call went badly yesterday. An executive asks what changed across all customers this week and gets an answer computed from states, not from reading thousands of documents. AI agents read a compact, shared state instead of rebuilding context from documents on every request.

Why links are the right training signal. Self-supervised learning needs a notion of "the same". Corrupting a record by cropping, masking or dropping words says that a record is the same as a damaged copy of itself; the model learns the record's wording, template and source. Predicting the next token teaches the model the language of the records. Neither says anything about the company. Links do: two records linked to the same customer in the same quarter are about the same situation, written by different people, in different systems, for different purposes, and what they share is the situation. This is the setting in which the Platonic Representation Theorem predicts convergence (§3). Links are not annotations: nobody creates them to train a model; they are metadata the systems keep in order to work. They are, however, a form of weak supervision, so we evaluate both on the records used in training and on records whose links were hidden.

3 Background

JEPAs and LeJEPA. A joint-embedding predictive architecture learns an encoder by making the embeddings of related views predictable from one another, without reconstructing the input [1, 20]. LeJEPA [2] replaces the usual heuristics against collapse with SIGReg, a regularizer that pushes the distribution of embeddings toward an isotropic Gaussian using random one-dimensional projections and the Epps–Pulley test. The loss pulls every view's projected embedding toward the mean of the global views and adds SIGReg: $\mathscr{L} = (1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$.

The Platonic Representation Theorem. Huh et al. [18] conjectured that representations learned by different models and modalities converge as they scale. Balestriero and LeCun [3] turn the conjecture into a theorem for JEPAs. We use five of its parts. (1) A JEPA trained with SIGReg learns a spectral embedding

of the *positive-pair graph*, the graph that says which views count as the same; the representation is set by that graph more than by the architecture. (2) Encoders trained on different modalities learn the same representation up to one rotation when their positive pairs are two observations of one scene (good views); otherwise their misalignment is bounded by how much the two graphs disagree about which scenes are alike. (3) That rotation is identified from a small number of anchor pairs by orthogonal Procrustes [30]. (4) Nuisance that the views do not share is discarded; the fraction of the embedding explained by the scene, ρ, measures how much of it is the world. (5) Next-token prediction can be read as a JEPA with bad views, parts of one document, which explains why language models align with vision only slowly as they scale. The paper validates the theory on images and captions. We test it on an organization.

4 Method

From records to states. We use customers as entities and calendar quarters as time windows, so a state is a (customer, quarter) pair. A record belongs to a state if its source system links it to the customer and its timestamp falls in the quarter. The links come from the systems themselves: a customer's dedicated chat channel, a customer header on an email, the CRM context of a recorded call, the organization of a support ticket, a customer field on an issue, the account of a CRM object and, for calendar events, the call they schedule. The encoder never reads these links; they only decide which records are positives. In our experiments we keep only linked records, so that every record can be scored against the ground truth; to see how the model places records without links, we hide the links of a quarter of the threads during training (§6.2).

Views. Each batch has 256 anchor records drawn uniformly, so states are weighted by their number of records, and each anchor gets two global and six local views, following LeJEPA's multi-crop recipe. With **good views**, each view is a record drawn at random, with replacement, from the anchor's state, so the views are usually *different records*, from any system. Each is trimmed to a contiguous span (50–100% of its tokens for global views, 15–50% for local views) with 15% of tokens masked. With **bad views**, all eight views are trimmed and masked copies of the anchor record itself: standard corruption-based self-supervision for text. Everything else is identical, so the only difference is the definition of "the same".

Encoder and training. So that the encoder learns from the company's data alone, with no pretrained language knowledge, we use a deliberately small model trained from scratch: unigrams and hashed bigrams (2^{18} buckets) feed a 192-dimensional embedding bag and a two-layer MLP with a 256-dimensional output, the representation we evaluate. A projector (1,024 hidden units, 128 outputs) is used only for the loss. We train with LeJEPA's loss ($\lambda = 0.05$, 256 random slices) for 1,500 steps with batches of 256 anchor records, a peak learning rate of 10^{-3}, 5% warm-up and cosine decay, using sparse Adam for the embedding table and AdamW elsewhere. One run takes three to six minutes on a CPU. The **next-token** baseline trains the same encoder on random prefixes of records with a softmax head over the whole vocabulary (6,061 words).

One space across systems. To test whether separately trained models agree, we split the systems into two families: messages people write to each other (email and chat) and systems of record (CRM, support, calls, calendar and issues). We train one encoder per family; each sees only its own family's records and never a cross-family pair. We whiten each encoder's output to 64 dimensions without labels and fit one orthogonal map between the two spaces by Procrustes on k anchor pairs, each a message and a record linked to the same state.

Stability over time: a rotation registry. When new data arrives, the encoder is retrained. Instead of re-embedding everything stored, we fit one rotation and one global scale from the new model's space to the old one on records both models have embedded, and rotate new queries into the old space. A registry of such rotations keeps every stored vector usable across model versions.

Access control, built in. Every record carries its *readers*, the people allowed to read it, which the source system already defines: the members of a private channel, the participants of an email thread, the teams that can open a ticket or a CRM record. Write D_u for the records reader u may read. A world model leaks across access boundaries in three ways, and each needs its own control:

(i) **Search** can return a record u may not read. Control: search only D_u, the usual fix in retrieval systems.

(ii) **Aggregation** can mix restricted records into what u sees: a customer's state vector is an average over its records, and one private escalation thread shifts it. Control: compute every state u sees from D_u only. Because states are averages, this costs one masked mean per state at read time.

(iii) **Training** can carry restricted content into the weights: an encoder trained on every record learns associations, such as a codename used only in leadership discussions and the customer it refers to, and reproduces them for anyone who queries it. Control: serve u an encoder trained only on records in D_u, with a vocabulary built from them.

With all three controls, every output u receives (search results, states, embeddings of u's own queries) is a function of D_u alone. Changing, adding or deleting any record outside D_u changes nothing u receives; this is *non-interference* [33], and here it holds by construction, not by estimate. In practice encoders are trained per access scope (a team, a role or a group of people who read the same records), and the first contribution of this paper makes that affordable: encoders trained on different but overlapping record sets agree up to a rotation (§6.5). The views of different people can therefore be expressed in shared coordinates, with each rotation fitted on records both scopes may read, and served only to people who may read all the data behind it. The result is one world model with many views, each view complete within its reader's rights and empty outside them.

5 Experimental setup

A simulated company. Era by Eon generates synthetic companies whose systems agree with each other, behind vendor-compatible interfaces and with exact ground truth [8, 11]. We ran seven of its public simulator images (Gmail, Google Calendar, Slack, Salesforce, Zendesk, Gong and Jira) with one seed, so all seven describe the same fictional company, Northwind Robotics, as of 31 October 2026 (*day 1*). We then advanced the simulators past the close of the next fiscal quarter, 30 January 2027 (*day 2*). An exporter pulls every text-bearing record through each system's API into one table of system, timestamp, author, thread, text and link fields. Table 1 summarizes the data. On day 1 the company has 78,048 records, of which 21,972 (28%) are linked to one of 21 active customers, spread over 180 customer-quarter states from late 2022 to late 2026. By day 2 the same customers have 7,990 new linked records, written in the three months after day 1 (most in the last calendar quarter of 2026, the rest in January 2027). Era generates records from one shared company graph, with text from templates and a pre-written text pack, part of it written by a language model, whose facts are filled in from the graph [8]. Real company text is messier, a point we return to in §9.

Baselines. Bad views and next-token prediction isolate the effect of the views: same text, same encoder, same budget. TF-IDF with latent semantic analysis (256 dimensions) is a classical text representation, and a

Table 1: The simulated company on day 1. A record is linked if its system ties it to one of the 21 active customers. Linked records define the states and the evaluation; all records can be embedded.

System	Records	Linked to a customer	Share linked
Slack (chat)	45,382	457	1%
Gmail (email)	19,713	14,160	72%
Salesforce (CRM)	5,077	1,136	22%
Zendesk (support)	4,117	4,117	100%
Gong (calls)	2,066	1,858	90%
Jira (issues)	1,206	77	6%
Google Calendar	487	167	34%
All	78,048	21,972	28%

Table 2: One encoder over all seven systems, measured on the training records (day 1). Mean ± standard deviation over three seeds for the JEPA objectives. Cross-system precision@10 for the same customer and the same state (customer and quarter); probe accuracy for the customer, trained on messages and tested on systems of record (M→R) and the reverse; ρ and R^2 are fractions of the whitened embedding explained by the state or by the system.

	Precision@10, other systems		Customer probe				
Representation	customer	state	M→R	R→M	ρ_{state}	R^2_{state}	R^2_{system}
Good views	**0.951** ± **0.003**	**0.390** ± **0.007**	**0.881** ± **0.003**	**0.927** ± **0.057**	**0.73** ± **0.01**	**0.61** ± **0.01**	0.04 ± 0.00
Bad views	0.136 ± 0.004	0.048 ± 0.002	0.228 ± 0.014	0.134 ± 0.013	0.16 ± 0.00	0.06 ± 0.00	0.08 ± 0.00
Next-token	0.132 ± 0.010	0.043 ± 0.003	0.248 ± 0.012	0.210 ± 0.024	0.18 ± 0.00	0.06 ± 0.00	0.06 ± 0.00
TF-IDF + LSA	0.346	0.105	0.358	0.383	0.14	0.06	0.05
Random encoder	0.132	0.055	0.188	0.150	0.17	0.05	0.04

randomly initialized encoder is a floor.

Metrics. *Cross-system precision@10*: for a record, the fraction of its ten nearest neighbours (cosine), restricted to records from *other systems* and other threads, that concern the same customer, or the same customer and quarter, averaged over 3,000 linked records; neighbours are searched among linked records. Restricting to other systems makes this a test of a shared state space rather than of a shared template. *Placement* (held-out records): each record is assigned to the training state whose mean embedding is nearest, and we report the share assigned to the right customer and to the right customer and quarter. *Cross-family probes*: a linear classifier trained to predict the customer from message embeddings and tested on record embeddings, and the reverse. ρ *and* R^2: on the whitened top 32 dimensions, the fraction of the embedding explained by the state, the customer or the system (between-group covariance with the within-group bias removed); ρ is the theorem's scene-explained fraction. *Alignment*: same-state recall@1 when messages retrieve records after the rotation, on held-out states, and mutual ten-nearest-neighbour agreement before it. *Time*: customer probes on new-quarter records, per-direction correlation between old and rotated new embeddings, and precision@10 of new queries against the old index.

6 Results

6.1 Good views absorb the company's links; bad views and next-token prediction do not

Figure 2 and Table 2 measure how each objective organizes the records it was trained on. With good views, 95.1% of a record's ten nearest neighbours from other systems concern the same customer, against 13.6%

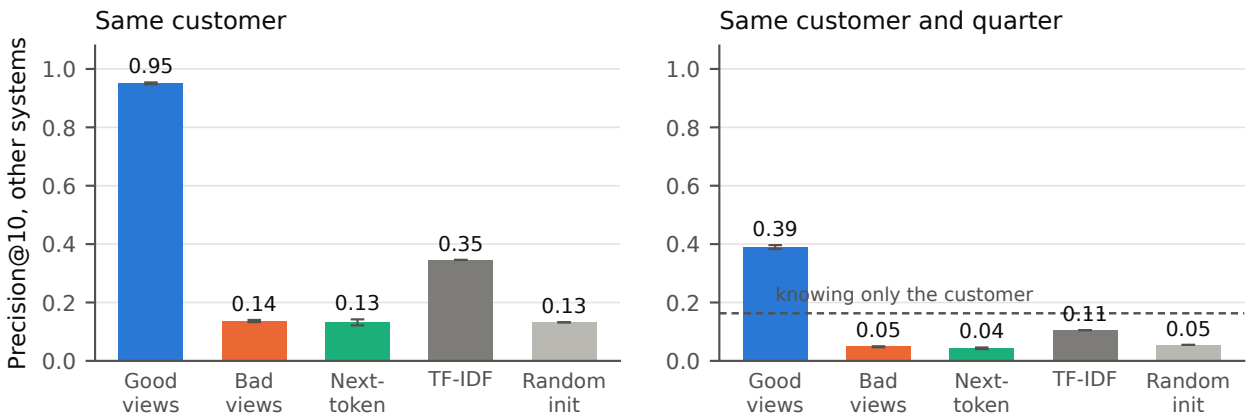


Figure 2: On the training records, only good views put records from different systems about the same situation together. Precision@10 among neighbours from *other* systems, for the same customer (left) and the same customer and quarter (right). Same text, encoder and budget for the three JEPA objectives; bars show the mean and standard deviation over three seeds. The dashed line is the most a model could reach by recognizing the customer alone.

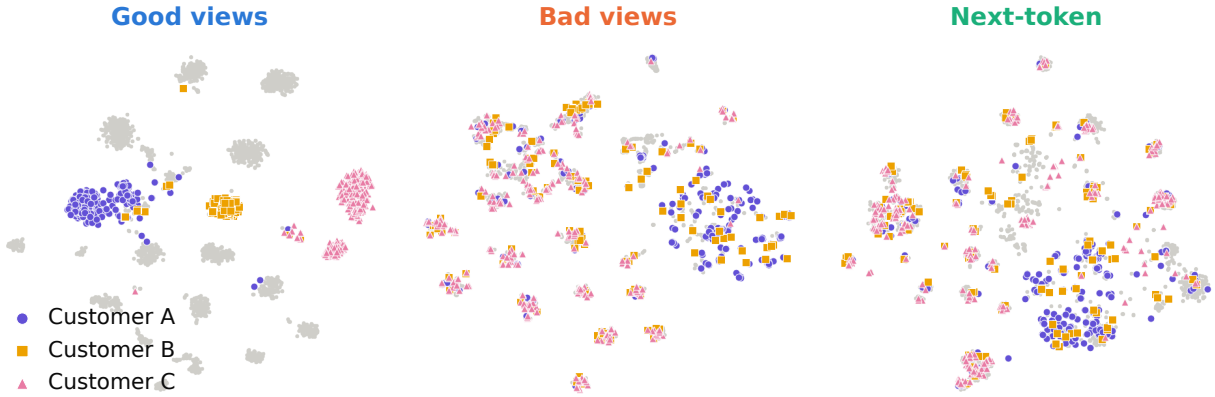


Figure 3: What each objective learns. Two-dimensional t-SNE maps of the same 3,000 training records from all seven systems. Three well-observed customers are highlighted; all other customers are grey. Only good views gather each customer's records, whatever system wrote them.

for bad views and 13.2% for next-token prediction, with the same text and model. TF-IDF reaches 34.6% and a randomly initialized encoder 13.2%: with this small encoder, corruption-based views and next-token prediction learn almost nothing about customers. For the state, customer and quarter together, good views reach 39.0% against about 4–5% for the other two objectives; a model that recognized only the customer could reach at most 16.3%, so the good-view space also separates a customer's quarters. A linear probe trained on messages names the customer of a CRM, support or call record 88% of the time with good views, and about a quarter of the time with the other objectives. The theorem's scene-explained fraction ρ is about 0.7 for good views and below 0.2 for every other representation, while the system that wrote a record explains about 4% of the good-view embedding. Figure 3 shows the difference: with good views, each customer's records from every system gather in a few tight regions; with the other objectives they are scattered among everyone else's.

These numbers are measured on the records used for training, with the same customer-quarter grouping that defined the positive pairs. They show how well each objective absorbs the links, not how well it generalizes. The next experiment tests generalization.

Table 3: Held-out records. A quarter of the threads (5,366 records) were hidden from training: their records were never a view or a prefix. Left: precision@10 among the hidden records, neighbours from other systems. Right: each hidden record is placed at the nearest training state (mean embedding); share placed with the right customer and with the right customer and quarter. One seed.

	Precision@10, other systems		Placed at the nearest state	
Representation	customer	state	customer	state
Good views	**0.703**	**0.187**	**0.755**	**0.262**
Bad views	0.110	0.028	0.187	0.043
Next-token	0.089	0.023	0.203	0.044
TF-IDF + LSA	0.263	0.065	0.527	0.136
Random encoder	0.095	0.026	0.319	0.083
Customer known, its busiest quarter			1.000	0.264
Largest customer / largest state			0.119	0.036

6.2 Records whose links were hidden find their place

We hid a quarter of the threads from training, 5,366 records whose links the model never saw, and trained each objective on the rest. We then placed each hidden record at the training state whose mean embedding is nearest (Table 3). Good views place 76% of the hidden records with the right customer and 26% with the right customer and quarter. TF-IDF places 53% and 14%, bad views 19% and 4%, and next-token prediction 20% and 4%. Knowing each record's customer and choosing that customer's busiest quarter would place 26% with the right state, about as many as good views do: on records the model has not seen, the gain is in finding the customer, not yet in telling that customer's quarters apart. Among the hidden records themselves, 70% of a record's ten nearest neighbours from other systems concern the same customer with good views, against 26% for TF-IDF. Records that carry no link at all, 72% of this company's records, are in a similar position, although they may differ in kind from linked records (most are chat messages); we cannot score them directly because they have no ground truth.

6.3 More than names

A natural worry is that the encoder merely learns to spot customer names, as embeddings of earnings calls mostly encode company identity [39]. We masked, in every record, the customers' names and their distinctive words, contact names, email addresses, web domains and record identifiers. The mask is deliberately aggressive: it changed 86% of linked records and also removes some ordinary words that happen to occur in a name. It keeps the names of the company's own employees, however, and an employee who works with only one customer can still give that customer away. We retrained the good-view encoder on the masked records (one seed). On the training records, same-customer precision across systems falls from 95% to 48%, and TF-IDF falls from 35% to 14% (Figure 4, right). Roughly half of the customer signal was names and identifiers; the rest comes from the remaining text, employees' names included. The masked model's state-level precision, 27%, stays above the 16.3% that knowing only the customer would give, so it still separates a customer's quarters.

6.4 Wrong links degrade gracefully

Real links are imperfect. Tickets get filed under the wrong organization, and duplicate customer records go unmerged. In the theorem, the damage is bounded by how much the wrong links change the positive-pair graph. We attached 25% and 50% of linked records to a random wrong state before training (one seed) and

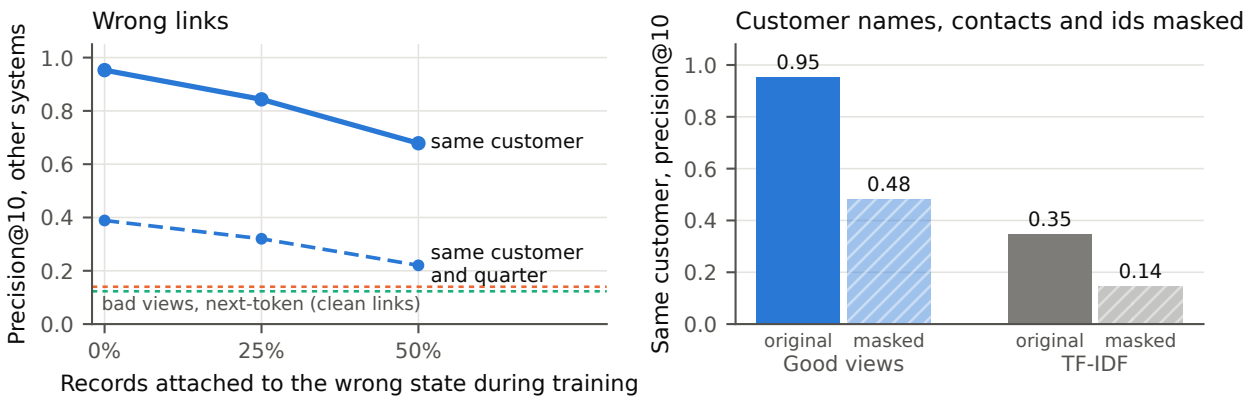


Figure 4: Robustness, on the training records. Left: good views trained with a share of records attached to a random wrong state, evaluated against the true states; dashed lines are bad views and next-token prediction with clean links. Right: same-customer precision before and after masking customer names, contacts, domains, email addresses and identifiers from every record.

evaluated against the true states (Figure 4, left). With 25% of the links wrong, same-customer precision is 84% (from 95% with clean links) and state precision 32% (from 39%). With 50% wrong, they are 68% and 22%. Even then, same-customer precision is far above that of bad views and next-token prediction trained on clean links (14% and 13%). Wrong links blur the space; they do not break it.

6.5 Separately trained encoders can be partly aligned by one rotation

We trained one encoder on messages and another on systems of record, with no pair crossing the two, and joined them with one orthogonal map (Figure 5). After a rotation fitted on 64 anchor pairs, a message retrieves a record of the same customer and quarter as its top match 20.7% of the time on states held out from the fit, against 3.8% by chance; with 2,000 anchors, 25.1%. Same-customer recall@1 is 41–50% against 14% by chance. With bad views or next-token prediction no rotation helps: same-state recall stays at 4–6%, near chance. Before any rotation, the mutual ten-nearest-neighbour agreement between the two spaces is 0.15 with good views and 0.02 with either alternative. The alignment is partial. A message finds a record of the same customer and quarter as its top match only about a fifth of the time; messages and systems of record observe different aspects of a customer's situation, while the theorem's guarantee assumes they observe the same state; and the two encoders share a vocabulary and were trained on the records of the test states, which are held out only from the rotation fit. Still, with good views a single rotation finds shared structure, and with the alternatives it finds none, consistent with the theorem's qualitative prediction.

6.6 New data and retraining

New records. A customer probe trained on day-1 embeddings assigns 72.6% of the 7,990 records written after training to the right customer with good views, against 41.0% for next-token prediction and 30.5% for bad views. The day-1 encoders never saw these records.

New models. We retrained the good-view encoder on all data through day 2, with the threads of 2,004 new records hidden from training, and fitted one rotation and one global scale from the new space to the old on 2,000 day-1 records embedded by both. Along the old space's ten leading principal directions, the old and rotated new embeddings correlate at 0.91–0.97; from the 33rd direction on, the mean correlation is 0.19 (Figure 7, left). The leading directions carry most of the variance (the first 16 carry 85%), so the rotation

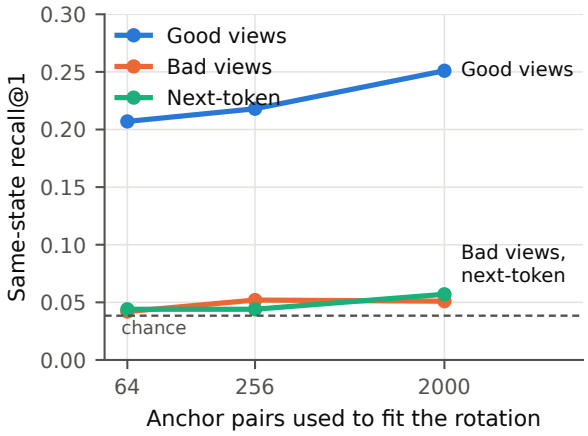


Figure 5: One rotation partly aligns two separately trained encoders, one for messages and one for systems of record, but only with good views. Same-state recall@1 of messages retrieving records, on states held out from the rotation fit.

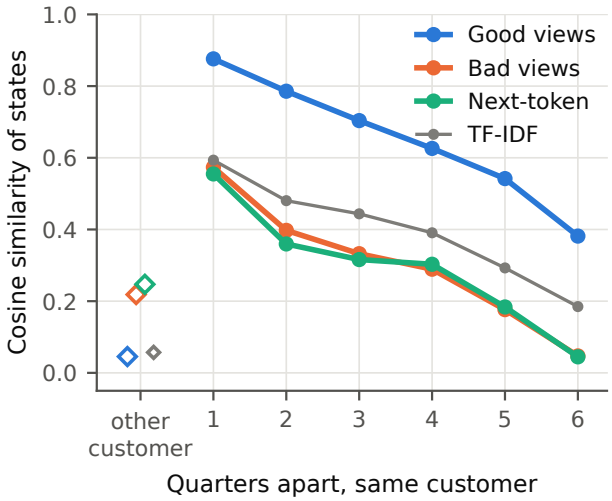


Figure 6: States persist and drift. Cosine similarity between centred state vectors of the same customer k quarters apart (lines) and of different customers in the same quarter (diamonds); states with at least five records.

explains 73% of the variance, but not the low-variance tail. We then used hidden new records, which neither model was trained on, as queries against the old index. Embedded by the retrained model and rotated, they retrieve the same customer with 79.4% precision@10, against 2.9% without the rotation and 71.6% when the old model embeds them (Figure 7, right).

Not every measure favours good views. In a second retraining test, scored in a whitened 64-dimensional space that weights every direction equally, a rotation fitted on 256 anchors explains less of a retrained good-view encoder than of a retrained bad-view encoder (R^2 of -0.22 against 0.28; mutual ten-nearest-neighbour agreement 0.36 against 0.59). A plausible reading is that bad-view encoders mostly encode surface features of the text, which any retraining reproduces, while the good-view encoder's leading directions, which carry the state, are stable and its tail is not. A rotation registry therefore preserves the part of the space that search relies on, not the whole space.

States persist and drift. We averaged each customer-quarter's records into a state vector (frozen day-1 encoders, all records through day 2) and compared states (Figure 6). With good views, a customer's states in consecutive quarters have a cosine similarity of 0.88, which falls smoothly to 0.38 six quarters apart, while different customers in the same quarter sit at 0.05. With bad views and next-token prediction, consecutive quarters are only 0.55–0.57 alike and different customers 0.22–0.25. In the good-view space a customer's state is persistent yet changes gradually: the raw material a model of dynamics needs.

6.7 Access control, built in

Who may read what. We gave every record its readers under a role-based policy for the simulated company (Appendix B). The CEO may read 100% of the customer-linked records, the support engineer 86%, the account executive 81%, the software engineer 22% and the financial analyst 9%.

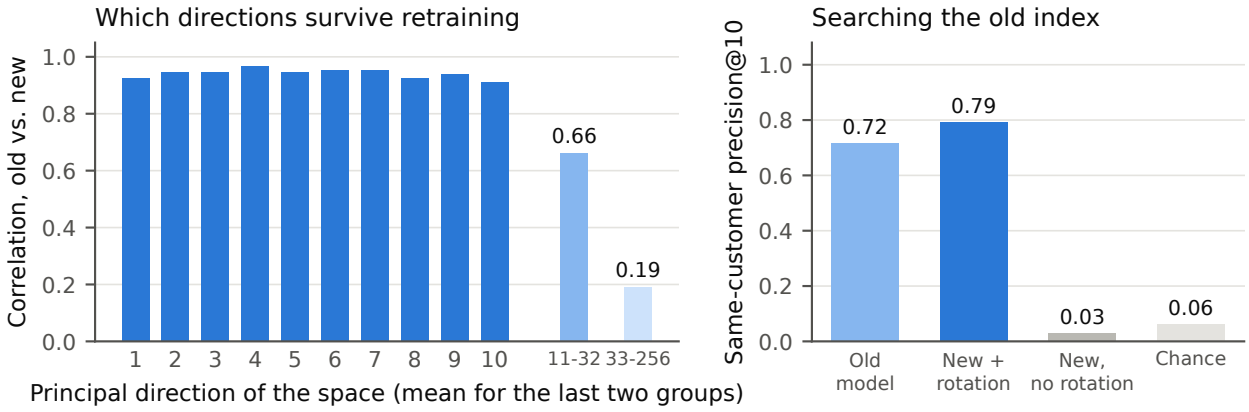


Figure 7: Retraining without re-embedding. Left: correlation between the old model's embeddings and the retrained model's rotated embeddings of the same records, along the old space's principal directions. Right: new-quarter records that neither model was trained on, searched against the old index.

Table 4: Planted secrets found by readers who may not read them. Codename search: the share of the ten records returned for a customer's codename that belong to that customer. Secret sentence: the share of the five customers most similar to the planted sentence that are the five canary customers. Chance is what a reader with no information would get. One seed.

		Codename search, precision@10		Secret sentence, precision@5	
Reader	Configuration	with secrets	without	with secrets	without
Support engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.80	0.40
	built in	0.16	0.16	0.40	0.40
Software engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.60	0.20
	built in	0.04	0.04	0.20	0.20

Leaks. We planted secrets in 150 CRM and call records of five customers, records that neither the support engineer nor the software engineer may read: a unique codename per customer, and one sentence, the same for all five, saying the customer is expected to churn to a competitor. We then measured what each of the two readers could find, and compared every configuration with the same configuration built without the secrets (Table 4, Figure 8, left). Without access control the secrets are in plain view: a codename search returns the right customer's records 100% of the time. Filtering search results and states to readable records, the usual fix, does not stop the leak. The encoder learned the codenames and the sentence from the restricted records and places them next to the right customers' *readable* records. A codename search still returns the right customer's records 100% of the time for the support engineer and 100% for the software engineer (0% and 0% without the secrets). The secret sentence puts 80% and 60% of the five canary customers among the five most similar customers (40% and 20% without). With access control built in, the readers' encoders never saw the restricted records, and the codenames are not even in their vocabularies: every result is the same with and without the secrets.

Non-interference, checked. We trained the support engineer's encoder twice, on the corpus with the planted secrets and on the corpus without them. Its training data was byte-for-byte identical, and so was every embedding it produced (largest difference: 0). The encoder trained on everything changed.

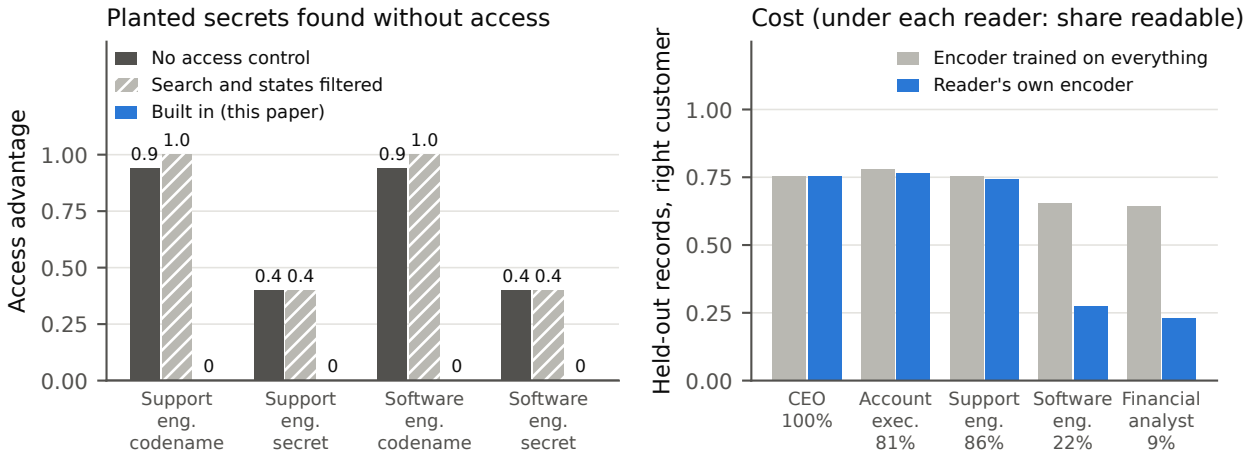


Figure 8: Access control, built in. Left: how much better than chance two readers without access find the planted secrets (codename search and secret sentence), with no access control, with search and states filtered to readable records, and with access control built in. Right: the cost, as the share of held-out readable records placed with the right customer by an encoder trained on everything and by the reader's own encoder; under each reader, the share of records they may read.

Cost. What a reader gives up depends on how much they may read (Figure 8, right). The CEO reads everything, so nothing changes. The account executive and the support engineer, who may read most records, lose little: their own encoders place held-out readable records with the right customer 76% and 74% of the time, against 78% and 75% for the encoder trained on everything. The software engineer and the financial analyst, who may read 22% and 9% of the records, lose much more (28% against 66%, and 23% against 65%): their encoders learn from few records, mostly from one or two systems. That gap is the price of the guarantee, and it is where the rest of the design matters: encoders trained per access scope rather than per person, a base encoder trained on what everyone may read, and rotations that place the views of different scopes in shared coordinates (§4). We have not yet tested these.

6.8 What does not work yet

Counting and forecasting. From each customer-quarter's state vector, the mean of its records' embeddings, we trained linear probes, holding out one customer at a time, to read four facts from Era's ground truth: the number of support tickets (four buckets), the worst ticket priority, whether a sales call took place and the deal stage. We also tried to predict the same facts for the *next* quarter (Appendix D). Two findings stand out. First, the good-view state vectors are poor at facts that amount to counting one system's records. For the ticket count, the worst priority and whether a call took place they do no better than the most common answer (0.59, 0.55 and 0.72 against 0.57, 0.57 and 0.71), while bad-view states read whether a call took place 97% of the time. One plausible reason is that good views are trained to ignore which system wrote a record, and these facts are about which system wrote one; another is that probes tested on unseen customers must extrapolate in a space organized by customer. We did not separate the two. The deal stage, the one fact about content, is read about equally well from every representation (34–43%, against 29% for the most common stage). Second, no representation forecasts the next quarter. With 165 transitions from 21 customers, every probe stays within a few points of the majority class (43–51% against 49%), far from simply copying this quarter's true facts (67%).

Reading the store with memory tokens. A Memorilla-style reader [9], sixteen query-conditioned memory tokens reading the 16 stored records retrieved for a question, improved accuracy on 144 templated test questions by 4.3 $\pm$ 1.0 points over a closed-book reader (mean $\pm$ standard deviation over four seeds). That is about six questions, within the binomial standard error of about four points. A store built from a generic pretrained sentence encoder did better, and adding the next quarter's records to the store lowered accuracy on questions about that quarter (Appendix E). We treat this result as inconclusive.

Both findings suggest the same design. A useful organizational world model probably needs two channels per record: a *state* vector, which says which situation a record belongs to and is shared across systems, and a *detail* vector, which keeps what the record specifically says and where it came from. Dynamics will need far more transitions than one simulated company provides.

7 Why this matters: from records to a model of the company

Most enterprise AI today is retrieval: given a question, find documents, read them, answer and forget. Every question pays the full cost of rebuilding the situation, and two questions about the same customer may rebuild it differently. A world model inverts this. The organization's state is maintained continuously as records arrive, and questions are answered from the state. The path from records to such a model has four steps, each useful on its own and each making the next possible: (1) a shared state space learned from the links an organization already keeps, the subject of this paper; (2) a memory that stores both the state and the detail of every record and answers from both; (3) a predictor of how states change, trained on the organization's own history of states, decisions and outcomes; and (4) planning, in which people and agents try decisions against the predictor before making them. Four properties make the first step practical, and the experiments above speak to each, with the caveats noted.

- **No annotation.** The training signal is the link structure the organization's systems already keep (§6.1, §6.2). Nobody labels anything, and wrong links blur the space instead of breaking it (§6.4).
- **Records without links find their place.** Records whose links were hidden during training are placed with the right customer by what they say (§6.2), and masking names shows that this is more than matching names (§6.3).
- **Every system, one space.** Encoders trained separately for different systems can be partly aligned by one rotation fitted on matched records (§6.5). A new tool, or an acquired company's chat, could join this way without retraining everything; we have not tested that directly.
- **Memory that survives upgrades.** One rotation lets a retrained encoder search the old index (§6.6). If that holds across many upgrades, which we have not tested, an organization's memory can compound across model versions instead of being rebuilt with each one.

Access control is part of the model. Enterprise systems took decades to encode who may see what. A world model that flattens those rules into one shared representation undoes that work, however carefully its search results are filtered: our experiments show restricted content leaking through the encoder's weights and through aggregated states even when every result shown is readable (§6.7). Building the three controls into the model instead keeps each person's view complete within their rights and empty outside them, with a guarantee that holds by construction. It also makes the model easier to trust and to audit: deleting a record removes its influence from every view that could see it, and from no other.

What changes for a company. When people and AI agents read the same state, coordination becomes measurable. Two teams whose records about the same customer drift apart in state space are misaligned before anyone notices. Hand-offs, onboarding and reviews start from the state instead of from a search. Because the state space is shared, what the organization knows accumulates in one place instead of in each person's head, and it stays usable when people leave.

Toward dynamics. A state space is the first half of a world model. The second half is a predictor over it: given a customer's state this quarter and an action, such as a discount, an escalation or a new owner, what is the likely next state? Predictors trained in a learned latent space are how world models are built for video and robotics [13, 20]. In an organization, the training data for such a predictor is the history of states, decisions and outcomes that the company already records. Our forecasting result shows that 165 quarterly transitions from one company are not enough; many companies, finer time windows and explicit actions will be needed. With a predictor, a company can ask "what happens if" before acting, and agents can plan against a model of the organization instead of against documents.

Personal world models. The same recipe applies to one person. Their email, calendar, messages and documents are different observations of their own projects and relationships, linked by people, threads and time.

Why every company will have an operational world model. Every company already has a world model: it is distributed across the heads of its people, partial, inconsistent and lost when they leave. Making it explicit changes three things. People and agents can *understand* how the company is functioning from one shared, permission-respecting state instead of reassembling it from documents. They can *learn* on it the way a person learns a company, from its own history of situations, decisions and outcomes, with a predictor over the state space as the learning substrate. And they can *improve* the company by trying a decision against the model before making it, and by seeing where the company's actual trajectory departs from the one it planned. The state space and the access control studied here are the foundation those three uses share.

8 Related work

Linked records as positives. Treating records that share a link as positive pairs is established: citations [7], authors and venues [40], product identifiers [27], foreign keys in relational databases [28], one bank client's transactions, locations and support calls [23], one patient's codes and notes [5] and, in industry, per-company embedding models fine-tuned on document links, co-access and chat threads.[1] The closest is the multimodal banking work, which trains one encoder per modality jointly with a CLIP-style objective [29]. We differ in defining states by entity *and* time window across a company's systems, training per-system encoders separately and joining them by a rotation, and testing the predictions of the Platonic Representation Theorem against bad-view and next-token controls.

JEPAs for text. LLM-JEPA applies a JEPA loss to curated pairs of text and code that describe the same thing [16], and Zuo et al. [41] apply JEPA with SIGReg to sections of a scientific paper. Our views come from organizational links, and our encoder is trained from scratch on one company.

[1] https://jxnl.co/writing/2025/03/06/fine-tuning-embedding-models-for-enterprise-rag-lessons-from-glean/

Convergence and alignment of representations. Beyond the Platonic hypothesis and theorem [3, 18], anchors and orthogonal maps join independently trained latent spaces [21, 24] and data silos [26], and orthogonal Procrustes has aligned word embeddings across decades [14]. Theory relates self-supervised representations to the spectrum of the positive-pair graph [15] and to the content that views share [36].

Compatible embeddings across model updates. Backward-compatible training makes a new model's embeddings comparable with an old index [32]; fitted Procrustes maps or affine adapters do so after training [22, 35]. Our rotation registry is an instance of the latter; for good-view encoders it preserves the leading directions of the space but not the tail.

Access control in machine learning. Information flow control for machine learning trains one expert module per security domain and combines only the experts a user may access, which gives non-interference for language models [33]. Permissioned LLMs attach parameter-efficient adapters to security domains and measure leakage with an access-advantage metric [19], and a position paper argues that enterprise models must be trained, and retrieve, only on content that every participant in an interaction may access [4]. Permission-aware retrieval, which filters search results by the reader's rights, is standard industry practice. We bring these ideas to a learned world model of an organization, where a third leak path appears, aggregation of records into states, and where the convergence of separately trained encoders lets per-scope models share one coordinate system.

Memory for language models. Memorilla compresses a document collection into a few query-dependent memory tokens for a frozen model [9], and xRAG compresses a retrieved document into one token [6].

Simulated organizations and enterprise world models. Era by Eon [11], OrgForge [10], TheAgentCompany [38] and CRMArena [17] simulate companies to evaluate agents; we use one to learn representations. Nair et al. [25] study learned world models of enterprise systems as predictors of state transitions and find that they falter when a deployment's configuration changes. Organizational memory has a long history in management research [37], and object-centric process mining relates events to the business objects they touch [34]. For individuals, general user models are built from a person's computer use and kept as text statements [31].

9 Limitations

- **One simulated company.** All results come from one Era company with 21 active customers. Era's text is cleaner and more regular than real company text, and its systems agree by construction. Replication on more simulated companies, and on real data, is the most important next step.
- **Weak supervision, evaluated partly on itself.** Links are not annotations, but they are supervision: the positive pairs come from the same customer-quarter grouping that most of our metrics score. Results on training records (§6.1, §6.3, §6.4) are therefore upper bounds; the held-out test (§6.2) is the fairer measure.
- **Weak alternatives.** With our small encoder, corruption-based views and next-token prediction learn little about customers on training records (no better than a random encoder there), and TF-IDF is the strongest alternative. Larger encoders may narrow the gap.

- **Small encoders, by design.** We use a bag-of-n-grams encoder trained from scratch so that nothing but the company's data shapes it. Transformers trained with the same recipe, or pretrained encoders fine-tuned on good views, may behave differently.
- **Seeds.** The main comparison uses three seeds; the held-out, alignment, time, masking, noise and access-control experiments use one.
- **Partial alignment.** Separately trained encoders agree on the leading directions, not on the tail, and messages and systems of record share only part of the state.
- **The masking test keeps employees' names**, which can identify a customer.
- **No dynamics yet, and no detail channel.** The state space does not forecast, and it is poor at facts about which system wrote a record (§6.8).
- **Access-control policy.** Our policy is role-based and built from what our export kept (system, channel privacy, direct-message membership, sender, organizer). Real deployments have finer rules (channel membership, email recipients, internal notes, record sharing), which plug into the same mechanism but were not tested. Per-scope encoders also cost one training run per scope, and very small scopes may not have enough records to train one.

10 Conclusion

Organizations already keep the training signal for a model of themselves. Records that different systems link to the same customer in the same window are good views of one situation. Training on them, with no annotation, yields a state space that places records from every system by the situation they describe, generalizes to records whose links it never saw, can be partly aligned across separately trained encoders, and lets a retrained encoder use an old index. Access control can be built into the same model, so that each person's view depends only on what they may read. Every company will come to run on an operational world model of this kind, one that people and agents can understand the company through, learn on and improve. At Sentra we are building it, and this paper lays down its first layers: the state space and the access control built into it. The next layers are a memory that keeps both state and detail, and a predictor of how states change and how actions change them.

Code and data. Code, the Era exporter, every result reported here and the scripts that draw the figures are released with this paper.[2] Era's simulators are free; the exact company can be regenerated from the seed and dates in the repository.

Acknowledgments. We thank Eon for making Era freely available. Experiments and drafting were assisted by an AI model (Claude, Anthropic); the author takes full responsibility for the content.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[2] Repository: `company-world-model` (link to be added on release).

[2] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.

[3] Randall Balestriero and Yann LeCun. Proving and improving the Platonic representation hypothesis with JEPAs. *arXiv preprint arXiv:2610.39302*, 2026.

[4] Shashank Shreedhar Bhatt, Tanmay Rajore, Khushboo Aggarwal, Ganesh Ananthanarayanan, Ranveer Chandra, Nishanth Chandran, Suyash Choudhury, Divya Gupta, Emre Kiciman, Sumit Kumar Pandey, Srinath Setty, Rahul Sharma, and Teijia Zhao. Position: Enterprise AI must enforce participant-aware access control. *arXiv preprint arXiv:2509.14608*, 2025.

[5] Tianxi Cai et al. Contrastive learning on multimodal analysis of electronic health records. *arXiv preprint arXiv:2403.14926*, 2024.

[6] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[8] Eon. Meet Era: Simulated companies to benchmark models and eval agents. https://www.eon.io/blog/era-build-and-test-ai-agents, 2026.

[9] Adibvafa Fallahpour, Parsa Idehpour, Nikita Mounier, Shirley Wu, Shayan Pardis, Vignesh Kothapalli, and Jure Leskovec. Memorilla: Latent semantic memory for LLMs. *arXiv preprint arXiv:2610.21375*, 2026.

[10] Jeffrey Flynt. OrgForge: A multi-agent simulation framework for verifiable synthetic organizational corpora. *arXiv preprint arXiv:2603.14997*, 2026.

[11] Benjamin Gruenbaum, Doron Porat, Assaf Natanzon, Roy Zavida, Chen Dinachi, and Or Itzahary. The Era by Eon benchmark: A generated enterprise estate with exact ground truth for benchmarking LLM agents. *arXiv preprint arXiv:2609.09853*, 2026.

[12] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[13] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[14] William L. Hamilton, Jure Leskovec, and Dan Jurafsky. Diachronic word embeddings reveal statistical laws of semantic change. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[15] Jeff Z. HaoChen, Colin Wei, Adrien Gaidon, and Tengyu Ma. Provable guarantees for self-supervised deep learning with spectral contrastive loss. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[16] Hai Huang, Yann LeCun, and Randall Balestriero. LLM-JEPA: Large language models meet joint embedding predictive architectures. *arXiv preprint arXiv:2509.14252*, 2025.

[17] Kung-Hsiang Huang et al. CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments. *arXiv preprint arXiv:2411.02305*, 2024.

[18] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The Platonic representation hypothesis. In *International Conference on Machine Learning (ICML)*, 2024.

[19] Bargav Jayaraman, Virendra J. Marathe, Hamid Mozaffari, William Shen, and Krishnaram Kenthapadi. Permissioned LLMs: Enforcing access control in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[20] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[21] Valentino Maiorca, Luca Moschella, Antonio Norelli, Marco Fumero, Francesco Locatello, and Emanuele Rodolà. Latent space translation via semantic alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[22] Lucas Maystre et al. When embedding models meet: Procrustes bounds and applications. *arXiv preprint arXiv:2510.13406*, 2025.

[23] Dzhambulat Mollaev, Ivan Kireev, Mikhail Orlov, Alexander Kostin, Ivan Karpukhin, Maria Postnova, Gleb Gusev, and Andrey Savchenko. Multimodal banking dataset: Understanding client needs through event sequences. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.

[24] Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà. Relative representations enable zero-shot latent space communication. In *International Conference on Learning Representations (ICLR)*, 2023.

[25] Jishnu Sethumadhavan Nair, Patrice Bechard, Rishabh Maheshwary, et al. Do enterprise systems need learned world models? The importance of context to infer dynamics. *arXiv preprint arXiv:2605.12178*, 2026.

[26] Keiyu Nosaka et al. Data collaboration analysis with orthonormal basis selection and alignment. *arXiv preprint arXiv:2403.02780*, 2024.

[27] Ralph Peeters and Christian Bizer. Supervised contrastive learning for product matching. *arXiv preprint arXiv:2202.02098*, 2022.

[28] Jakub Peleška and Gustav Šír. Task-agnostic contrastive pretraining for relational deep learning. *arXiv preprint arXiv:2506.22530*, 2025.

[29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[30] Peter H. Schönemann. A generalized solution of the orthogonal Procrustes problem. *Psychometrika*, 31 (1):1–10, 1966.

[31] Omar Shaikh, Shardul Sapkota, Shan Rizvi, Eric Horvitz, Joon Sung Park, Diyi Yang, and Michael S. Bernstein. Creating general user models from computer use. *arXiv preprint arXiv:2505.10831*, 2025.

[32] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards backward-compatible representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[33] Trishita Tiwari, Suchin Gururangan, Chuan Guo, Weizhe Hua, Sanjay Kariyappa, Udit Gupta, Wenjie Xiong, Kiwan Maeng, Hsien-Hsin S. Lee, and G. Edward Suh. Information flow control in machine learning through modular model architecture. In *USENIX Security Symposium*, 2024.

[34] Wil M. P. van der Aalst. Object-centric process mining: Dealing with divergence and convergence in event data. In *Software Engineering and Formal Methods (SEFM)*, 2019.

[35] Harshil Vejendla. Drift-adapter: A practical approach to near zero-downtime embedding model upgrades in vector databases. *arXiv preprint arXiv:2509.23471*, 2025.

[36] Julius von Kügelgen, Yash Sharma, Luigi Gresele, Wieland Brendel, Bernhard Schölkopf, Michel Besserve, and Francesco Locatello. Self-supervised learning with data augmentations provably isolates content from style. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[37] James P. Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991.

[38] Frank F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

[39] Ding Yu, Zhuo Liu, and Hangfeng He. Same company, same signal: The role of identity in earnings call transcripts. *arXiv preprint arXiv:2412.18029*, 2024.

[40] Yu Zhang et al. Metadata-induced contrastive learning for zero-shot multi-label text classification. *arXiv preprint arXiv:2202.05932*, 2022.

[41] You Zuo, Éric de la Clergerie, and Benoît Sagot. Asymmetric within-document predictive learning for scientific document representation. *arXiv preprint arXiv:2608.28625*, 2026.

A Experimental details

Data export. The seven simulators run as Docker images (`erabyeon/*`) with seed 42, the simulated date set to 31 October 2026 and high fidelity. The exporter reads every system through its REST interface: Slack messages, Gmail messages, Google Calendar events, Salesforce objects (accounts, opportunities, tasks, cases and notes), Zendesk tickets and comments, Gong call transcripts and Jira issues and comments. Day 2 is the simulators' second state, after the fiscal quarter that closes on 30 January 2027. Texts are lower-cased and tokenized into words; each record keeps its first 96 tokens, and the vocabulary keeps words that occur at least twice among the linked records (6,061 words on day 1; it is built before any held-out split, from text only). All experiments use linked records only.

Hyperparameters. Encoder: embedding bag of 2^{18} buckets $\times$ 192, LayerNorm, Linear(192, 512), GELU, Linear(512, 256). Projector: Linear(256, 1024), BatchNorm, GELU, Linear(1024, 128). Loss: $(1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$, $\lambda = 0.05$; SIGReg with 256 random directions and 17 integration knots on $[-5,5]$. Views: 2 global (50–100% contiguous span) and 6 local (15–50%), 15% token masking. Optimization: 1,500 steps (1,000 for the per-family encoders), batch 256, peak learning rate 10^{-3}, 5% linear warm-up, cosine decay, AdamW (weight decay 0.05) for dense parameters and sparse Adam for the embedding bag. Next-token baseline: 1,024 random prefixes per batch, softmax over the whole vocabulary.

Evaluation details. Cross-system precision@10 excludes neighbours from the query's own system and thread and averages over 3,000 random linked records. Probes are multinomial logistic regressions. ρ and R^2 use the top 32 dimensions after whitening, with the between-group covariance corrected for its finite-sample bias. For the alignment test, the 126 states observed by both families are split in half; anchors come from one half, test pairs (1,000) from the other. The per-direction analysis of the retrained encoder fits one rotation and one global scale on 2,000 day-1 records embedded by both models and tests it on 6,000 others, in the centred (not whitened) space; its variance-weighted R^2 is 73% (88% on the top 8 directions). The second retraining test in §6.6 is a separate run: a retrained encoder (different seed, no hidden records), a rotation fitted on 256 anchors in a whitened 64-dimensional space without scale. For the held-out test, threads are drawn at random among linked records; TF-IDF is fitted on the training records only. For the state probes and Figure 6, TF-IDF is refitted on the day-2 corpus.

Masking. Customer account and organization names, the distinctive words in them (words shared by several customer names, such as "payments", are kept), customer web domains, contact names, email addresses and identifiers of records, cases, requests and long numbers are replaced with one placeholder token before tokenization. Names of the company's own employees are kept.

B Access-control policy

Readers are derived from each record's system, channel and sender with the role-based policy below (`cwm/acl.py`); the second table gives the share of customer-linked records each evaluated reader may read, overall and by system. For the leak tests, a unique codename and the sentence "Privately we expect them to churn to a competitor next quarter." were appended to 150 training records of five customers in Salesforce and Gong, which neither the support engineer nor the software engineer may read. Each reader's encoder is trained with the same recipe as the main comparison (good views, one seed, a quarter of the threads held out), on the records that reader may read and with a vocabulary built from them.

Table 5: The access policy for the simulated company, and what each evaluated reader may read.

Records	Who may read them
Slack, public channels (including customer channels)	everyone
Slack, private channels (leadership, exec-staff)	leadership
Slack, direct and group messages	their members
Jira (issues)	everyone
Zendesk (support tickets)	support, success, engineering, product, leadership
Salesforce (CRM)	sales, success, finance, leadership
Gong (call transcripts)	sales, success, product, leadership
Gmail	the sender, plus sales, success, support, leadership
Google Calendar	the organizer, plus leadership

Reader	all	slack	gmail	zendesk	salesforce	gong	jira	calendar
CEO	100%	100%	100%	100%	100%	100%	100%	100%
Account executive	81%	100%	100%	0%	100%	100%	100%	59%
Support engineer	86%	100%	100%	100%	0%	0%	100%	25%
Software engineer	22%	100%	1%	100%	0%	0%	100%	1%
Financial analyst	9%	100%	2%	0%	100%	0%	100%	0%

C All results of the main comparison

Table 6: Per-seed results of the main comparison on the training records: cross-system precision@10, customer probes and ρ.

Objective	Seed	Same customer	Same state	M→R	R→M	ρ_{state}
Good views	0	0.953	0.389	0.883	0.978	0.72
Good views	1	0.952	0.384	0.878	0.937	0.74
Good views	2	0.947	0.397	0.881	0.866	0.73
Bad views	0	0.140	0.050	0.244	0.142	0.17
Bad views	1	0.137	0.049	0.221	0.141	0.16
Bad views	2	0.132	0.046	0.218	0.119	0.17
Next-token	0	0.123	0.041	0.261	0.184	0.18
Next-token	1	0.129	0.042	0.243	0.213	0.18
Next-token	2	0.143	0.046	0.240	0.232	0.18

D State tracking and forecasting

For each customer-quarter, the state vector is the mean of the L2-normalized embeddings of its records, from the frozen day-1 encoders applied to all records through day 2. Probes are standardized, reduced to 32 principal components and fitted by logistic regression, holding out one customer at a time, so that no probe can memorize a customer. "Now" predicts a quarter's facts from its own state; "next" predicts the following quarter's facts from the current state (165 transitions). The majority class is the share of the most common answer. Oracle persistence copies the current quarter's true facts and therefore reads the ground truth directly.

Table 7: Accuracy of linear probes on state vectors, holding out one customer at a time.

	This quarter					Next quarter				
Representation	tickets	priority	call	stage	mean	tickets	priority	call	stage	mean
Good views	0.59	0.55	0.72	0.43	0.57	0.32	0.35	0.68	0.35	0.43
Bad views	0.72	0.71	0.97	0.41	0.70	0.48	0.49	0.62	0.35	0.48
Next-token	0.71	0.68	0.94	0.34	0.67	0.55	0.52	0.64	0.27	0.49
TF-IDF + LSA	0.70	0.67	0.82	0.41	0.65	0.52	0.53	0.60	0.39	0.51
Random encoder	0.64	0.63	0.85	0.36	0.62	0.49	0.46	0.68	0.29	0.48
Majority class	0.57	0.57	0.71	0.29	0.54	0.51	0.51	0.65	0.30	0.49
Oracle persistence			—			0.71	0.59	0.69	0.70	0.67

E Reading the store with memory tokens

Each linked record is stored once as a row: its embedding mapped to 112 dimensions by PCA whitening, a 16-dimensional sinusoidal time code and a one-hot code of its source system. A Memorilla-style module [9] turns a question, embedded by the store's encoder, into 16 query-conditioned slots that attend, through two Perceiver-style cross-attention blocks, either over the whole store or over the 16 rows most similar to the question. The slots are averaged and matched against fixed embeddings of the 18 possible answers from a pretrained sentence encoder (all-MiniLM-L6-v2), so the reader is a classifier with a frozen output layer rather than a language model. Questions are generated from Era's ground truth for each customer-quarter: the number of support tickets, the worst ticket priority, whether a sales call happened and the deal stage. Customer-quarters are split 70/10/20 into training, validation and test (36 test customer-quarters, 144 test questions). The module is first pre-trained for 250 steps to recover what the store holds about a customer and date, including the share of records from each system and their volume, then trained on the questions for 12 epochs, keeping the checkpoint that does best on validation. With the generic store, questions are also embedded by the generic encoder, which changes the closed-book baseline.

Table 8: Accuracy of the memory reader on the 144 test questions (mean over seeds); the answer layer is frozen.

Store	Reader	Seeds	Accuracy
Good-view encoder	closed book (no store)	4	0.618
	16 tokens over the 16 retrieved rows, pre-trained	4	0.661
	16 tokens over the whole store, pre-trained	1	0.639
	16 tokens over the whole store, not pre-trained	1	0.639
Pretrained sentence encoder	closed book (no store)	3	0.632
	16 tokens over the 16 retrieved rows, pre-trained	3	0.701
Questions about the next quarter (172 questions, majority 0.436)			
Good-view encoder	closed book	4	0.547
	retrieved rows, store not updated	4	0.599
	retrieved rows, store updated	4	0.561